L06. Multilevel Logic

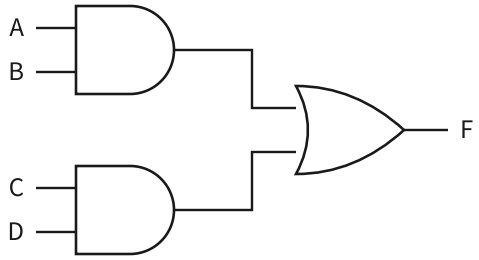
Lecture 06

6.1 SoP / PoS 를 다른 게이트로 구현하기

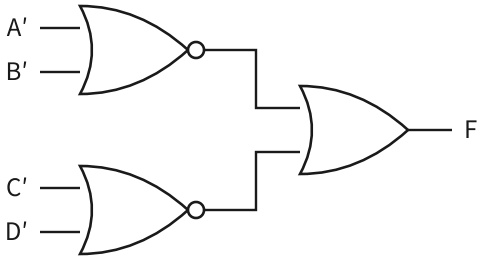
SoP/PoS 구현은 AND-OR 외에 NOR-OR, NAND-NAND, OR-NAND 도 됨.

AND-OR 형태 SoP 네트워크를 다른 게이트 조합으로 바꿔 보기. 예: F = AB + CD.

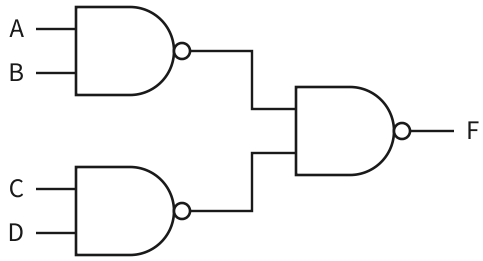
(A) AND-OR



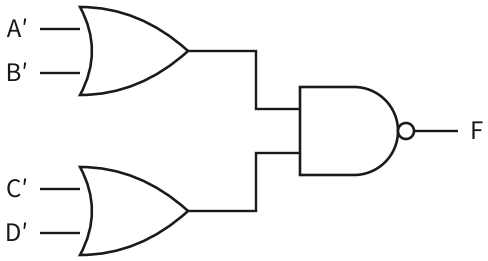
(B) NOR-OR



(C) NAND-NAND



(D) OR-NAND



F = AB + CD 를 네 가지 2단 회로로 구현한 것. 네 회로는 모두 같은 함수다.

(A) AND-OR	기본형.
(B) NOR-OR	$AB = (A' + B')'$ 이므로 AND 를 입력 보수의 NOR 로 대체.
(C) NAND-NAND	첫 단 출력이 $(AB)'$, 둘째 단 NAND 에서 $((AB)'(CD)')' = AB + CD$ 가 나옴.
(D) OR-NAND	$(AB)' = A' + B'$ 이므로 NAND 를 입력 보수의 OR 로 대체.

6.2 다단 논리 (Multilevel logic)

> 다단 논리: 어떤 경로에 게이트가 2개를 넘게 있는 논리. 2단 논리는 AND-OR 두 단이 전부.

예

X = ADF + AEF + BDF + BEF + CDF + CEF + G (SoP, 2단 논리)

X = [(A+B+C)(D+E)F] + [G] (sum-of-products-of-sums, 다단 논리)

2단 회로에는 AND 6개(입력 3개씩)와 7-입력 OR 하나가 필요함. 다단으로 바꾸려면 factoring 을 쓰는데, 그러면 단 수는 늘어남.

왜 다단 논리인가

게이트 효율(면적)	다단이 2단보다 게이트 효율이 좋은 경우가 있음. 2단 회로는 fan-in이 커서 그렇다.
속도(지연)	다단이 더 빠른 경우도 있지만 항상은 아님.
최적화	다단은 2단보다 최적화가 어렵다.
다단 단순화 기법	factoring, decomposition, extraction. 최적의 다단 논리를 보장하는 알고리즘은 없음.

1. Factoring

> Factoring: SoP 식에서 공통 인수를 묶어 다단 식으로 바꾸는 것. 2단 식이 다단 식이 됨.

예

	식	리터럴
factoring 전	F = AC + AD + BC + BD + E	9
factoring 후	F = (A + B)(C + D) + E	5

2. Decomposition

> Decomposition: 식 하나를 새 식 여러 개로 쪼개서 리터럴 수를 줄이는 것. X 와 X′ 둘 다 다룸.

예

	식	리터럴
전	F = ABC + ABD + A′C′D′ + B′C′D′	12
후	F = AB(C+D) + (A′+B′)C′D′ = AB(C+D) + (AB)′(C+D)′ = XY + X′Y′	8

새 식은 X = AB, Y = C + D. 그러면 F = XY + X′Y′, 즉 X 와 Y 의 XNOR.

3. Extraction

> Extraction: 여러 출력 함수가 같이 쓰는 공통 부분 함수를 뽑아내는 것.

예

	식	리터럴 / 게이트
전	F = (A+B)CD + E, G = (A+B)E′, H = CDE	11 리터럴, 게이트 8개 (2입력 AND/OR, 인버터 제외, 출력 간 공유 전)
후	F = XY + E, G = XE′, H = YE (X = A+B, Y = CD)	11 리터럴, 게이트 6개 (같은 기준, X와 Y 공유)

6.3 다음 강의와의 관계

다음 강의(L07)에서는 다단 회로에서 경로마다 다른 게이트 지연이 만드는 글리치와 hazard 를 다룸.